

TanStack Query — Install & Setup

From `npm install` to Ready-to-Use in 5 Minutes

CSCI 39548 — Practical Web Development

What Is TanStack Query (In One Sentence)

TanStack Query handles fetching data from your backend for you — including loading states, error states, caching, and refetching — so your components stop juggling all of that by hand.

Think of the things you wrote by hand in A2: `useState` for loading, another for error, a third for data, a `useEffect` with a `fetch` inside. TanStack Query replaces all that with one hook.

This deck only **installs** the library. Using `useQuery` is the next deck.

What You Need Before You Start

Confirm three things before you `npm install`:

1. A **working React + Vite project**. If `npm run dev` opens your app at `localhost:5173`, you're good. If not, fix that first.
2. **React 18 or later**. Check `package.json` — if `"react": "^18.x.x"` or higher, you're fine. A3 used React 19 by default.
3. A **terminal open in your project folder** — the folder with `package.json` for your frontend. If you have a `frontend/` subfolder, `cd` into it first.

Install Command

From inside your React project folder:

```
npm install @tanstack/react-query
```

One command. ~10 seconds. Total package size is small (~30 KB gzipped to your final bundle).

If you see a warning about peer dependencies (React version), read it. If it's a real version mismatch, fix React first. If it's just a notice, ignore it.

What npm install Actually Does

Three things happen:

1. **Downloads** the package from the npm registry into `node_modules/@tanstack/react-query/`
2. **Updates** `package.json` — a new line appears under "dependencies":

```
"@tanstack/react-query": "^5.x.x"
```

3. **Updates** `package-lock.json` with the exact version + every sub-dependency

Commit both `package.json` and `package-lock.json`. Never commit `node_modules/`.

Verify the Install Worked

Three quick checks:

1. **Install output** says added N packages (not npm ERR!)
2. `package.json` now lists "`@tanstack/react-query`" under dependencies
3. `ls node_modules/@tanstack/react-query` shows files including `package.json`, `build/`, `README.md`

All three pass = the library is installed. Now we need to **configure** it.

(Optional) Install the DevTools

A separate package — a floating debug panel showing every query, its state, cached data, and refetch history.

```
npm install @tanstack/react-query-devtools
```

- ▶ Only loads when you explicitly use it in your code
- ▶ Only shows up in development — Vite strips it from production builds
- ▶ No cost to installing it

Strongly recommended for class. It makes “where is my data?” answerable in one click.

Now Configure: the `QueryClient`

A package being installed isn't enough — you have to **tell React to use it**. TanStack Query needs two things:

- ▶ A **`QueryClient`** — one instance, created once at startup. Holds the cache, default settings, and is the brain of the library.
- ▶ A **`QueryClientProvider`** — a React component that wraps your entire app and gives every component access to the `QueryClient`.

Without these two, calling `useQuery` anywhere crashes with:

```
"No QueryClient set, use QueryClientProvider to set one."
```


Open src/main.tsx

The entry point of every Vite + React project. It usually looks like:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.tsx";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

We'll add: **two imports**, **one line creating the client**, and **one component wrapping** `<App />`. Total: ~5 new lines.

Edit 1: Add the Imports

At the top of `src/main.tsx`, with the other imports:

```
import { QueryClient, QueryClientProvider }  
  from "@tanstack/react-query";
```

`QueryClient` is the class you'll instantiate. `QueryClientProvider` is the component you'll wrap your app in.

Both come from the same package — one import line, two named imports.

Edit 2: Create the QueryClient Instance

Below the imports but **above** `createRoot(...)`, add one line:

```
const queryClient = new QueryClient();
```

- ▶ Creates a single `QueryClient` for your entire app
- ▶ Empty `()` = use all default settings (sensible for almost every app)
- ▶ You can pass an options object later to tune caching, refetching, etc.

Note: One client per app. Never create a new `QueryClient` inside a component — that resets the cache every render.

Edit 3: Wrap `<App />` in the Provider

Before:

```
<StrictMode>  
  <App />  
</StrictMode>
```

After:

```
<StrictMode>  
  <QueryClientProvider client={queryClient}>  
    <App />  
  </QueryClientProvider>  
</StrictMode>
```

The Provider passes the `queryClient` down through React Context, so any component can use TanStack Query hooks.

The Finished main.tsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { QueryClient, QueryClientProvider }    //+
  from "@tanstack/react-query";              //+
import "./index.css";
import App from "./App.tsx";

const queryClient = new QueryClient();        //+

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>  {/***/}
      <App />
    </QueryClientProvider>                    {/***/}
  </StrictMode>
);
```

(Optional) Add the DevTools Panel

If you installed `@tanstack/react-query-devtools`, add:

Import:

```
import { ReactQueryDevtools }  
  from "@tanstack/react-query-devtools";
```

Inside the Provider, after `<App />`:

```
<QueryClientProvider client={queryClient}>  
  <App />  
  <ReactQueryDevtools initialIsOpen={false} />  
</QueryClientProvider>
```

`initialIsOpen={false}` = collapsed by default. A small floating icon appears bottom-corner; click to expand. Only renders in development.

Verify the Setup Worked

Three checks:

1. Run `npm run dev`. App starts on `localhost:5173` with no errors. If you see a red error mentioning `@tanstack/react-query`, re-check the imports.
2. Open the app in browser → DevTools → Console. **No red errors** mentioning `QueryClient` or `QueryClientProvider`.
3. If you installed devtools: look at the **bottom corner** of the browser window. A small flower icon appears. Click it — a panel slides up showing “No queries.” That panel being there proves the setup works.

All three pass = **you are ready to use TanStack Query.**

What You Have Now

- ▶ TanStack Query installed in `node_modules` and recorded in `package.json` ✓
- ▶ A single `QueryClient` instance created at app startup ✓
- ▶ `QueryClientProvider` wrapping your entire app ✓
- ▶ (Optional) The devtools panel installed and rendering in dev ✓

Any component in your app can now call `useQuery` without doing any additional setup. The library is live.

What's next: the main TanStack Query deck covers `useQuery` (reading data), `useMutation` (writing data), query keys, the cache lifecycle, and the patterns you'll use in A4.

Common Install Errors

- ▶ Cannot find module '@tanstack/react-query' → install didn't happen. Re-run `npm install @tanstack/react-query` in the project folder.
- ▶ No `QueryClient` set at runtime → `<QueryClientProvider>` doesn't wrap the component calling `useQuery`. Confirm it's in `main.tsx` wrapping `<App />`.
- ▶ **DevTools panel doesn't appear** → import path must be `@tanstack/react-query-devtools` (not `@tanstack/query-devtools`), and `<ReactQueryDevtools />` must be inside the `Provider`.
- ▶ Module not found **after install** → stop dev server (`Ctrl+C`), restart. Vite caches module resolutions.

Rule of thumb: if something fails, stop the server, delete `node_modules/`, run `npm install`, restart. Fixes 90% of weird errors.